

ATIVIDADE FINAL

Praticando Branches e Merge com Git

Práticas DevOps em Projetos Web

1. Objetivo

Praticar o uso de branches, commits e merge no Git/GitHub, simulando o trabalho de uma equipe de desenvolvimento.

2. Criando o projeto

Crie um novo repositório no GitHub chamado: **atividade-git-final**

O repositório deverá ser iniciado com um arquivo README.md. No README, coloque:

- Nome do aluno
- Nome da atividade
- Uma breve descrição do projeto

Faça o primeiro commit com uma mensagem clara, por exemplo: **Criar projeto inicial**

3. Criando a primeira branch

A partir da main, crie uma nova branch chamada: **feature-equipe**

Nessa branch, crie um arquivo chamado equipe.txt. Dentro dele, coloque algumas informações fictícias sobre uma equipe.

Exemplo:

Equipe do projeto

Desenvolvedor: João

Designer: Maria

Analista: Pedro

Faça o commit. Depois, volte para a main e faça o merge da feature-equipe.

4. Criando uma segunda branch

A partir da main, crie: **feature-requisitos**

Crie um arquivo chamado requisitos.txt e coloque pelo menos 5 requisitos fictícios para um sistema.

Exemplo:

1. O sistema deverá permitir cadastro de usuários.
2. O sistema deverá permitir login.
3. O sistema deverá permitir alterar dados.
4. O sistema deverá permitir consultar informações.
5. O sistema deverá possuir controle de acesso.

Faça o commit e depois faça o merge dessa branch na main.

5. Criando uma terceira branch

A partir da main, crie:

feature-backlog

Crie um arquivo chamado backlog.txt e liste pelo menos 5 tarefas que poderiam ser realizadas em um projeto.

Exemplo:

- Criar tela de login
- Criar cadastro de usuários
- Criar tela inicial
- Implementar recuperação de senha
- Realizar testes

Faça o commit e depois faça o merge na main.

6. Desafio Extra

Agora será simulada uma situação em que duas alterações diferentes foram feitas na mesma parte do projeto.

1. Crie uma branch chamada: **feature-descricao**
2. Nessa branch, altere o README.md adicionando uma descrição para o projeto.
3. Faça o commit, mas NÃO faça o merge ainda.
4. Volte para a main e altere a mesma parte do README.md, criando uma descrição diferente.

5. Faça outro commit.
6. Tente fazer o merge da feature-descricao na main. O Git deverá identificar um conflito.

7. Resolvendo o conflito

Ao encontrar o conflito, o aluno deverá:

1. Identificar o conflito apresentado pelo Git.
2. Escolher qual informação deverá permanecer.
3. Resolver o conflito.
4. Finalizar o merge.
5. Verificar o histórico de commits.

8. Entrega

Enviar apenas o link do repositório no GitHub. O professor deverá conseguir visualizar as branches utilizadas, os commits, os merges, o histórico do projeto e a resolução do conflito.

9. Checklist

- ☐ Projeto criado no GitHub
- ☐ Branch main utilizada
- ☐ Branch feature-equipe criada e mesclada
- ☐ Branch feature-requisitos criada e mesclada
- ☐ Branch feature-backlog criada e mesclada
- ☐ Histórico de commits revisado

Os demais itens são opcionais caso o aluno tenha mostrado facilidade na primeira parte da atividade

- ☐ Branch feature-descricao criada
- ☐ Conflito provocado e identificado
- ☐ Conflito resolvido
- ☐ Merge finalizado